Cache Control Strategies in Next.js

---

1) The 4 Layers of Caching in Next.js

Think of caching in four distinct buckets:

1. **Request Memoization (per request)**

- Happens automatically inside a single request lifecycle
- Same `fetch()` call deduplicated

```
await fetch('/api/products')
await fetch('/api/products') // reused automatically
```

Scope: One request only **Use case:** Avoid duplicate work in RSC trees

---

2. **Data Cache (persistent fetch cache)**

This is the most important layer.

```
await fetch('https://api.example.com/products', {
  cache: 'force-cache'
})
```

Options:

- `force-cache` → cache forever (default in RSC)
- `no-store` → always fetch fresh
- `revalidate: 60` → ISR-style

```
await fetch(url, {
  next: { revalidate: 60 }
})
```

Scope: Across requests (server-side persistent cache)

---

3. **Full Route Cache (HTML + RSC payload)**

Next.js caches the **entire rendered output** of a route.

Triggered when:

- Page is static

- Or uses revalidation

```
export const revalidate = 60
```

Effect:

- Whole page served from cache
- No server execution until revalidation

---

4. **Router Cache (client-side)**

- Cached in browser memory
- Enables instant navigation between routes

```
<Link href="/products" />
```

Scope: Per user session

---

2) Core Cache Control Strategies

Now the practical part—how you actually control behavior.

Strategy 1: Fully Static (Maximum Performance)

```
export const revalidate = false
```

or:

```
fetch(url, { cache: 'force-cache' })
```

Use when:

- Marketing pages
- Docs
- Rarely changing data

Strategy 2: ISR (Time-based Revalidation)

```
export const revalidate = 60
```

or per fetch:

```
fetch(url, {
  next: { revalidate: 60 }
})
```

Behavior:

- First request → cached
- After 60s → stale served, background refresh

Use when:

- Product listings
- Blogs
- Dashboards with tolerable staleness

Strategy 3: On-Demand Revalidation (Event-driven)

Instead of time, you invalidate explicitly.

```
import { revalidateTag } from 'next/cache'

revalidateTag('products')
```

Attach tag:

```
fetch(url, {
  next: { tags: ['products'] }
})
```

Use when:

- Admin adds product
- CMS update
- Webhook triggers refresh

This is the **most production-grade strategy**.

Strategy 4: Fully Dynamic (No Cache)

```
export const dynamic = 'force-dynamic'
```

or:

```
fetch(url, { cache: 'no-store' })
```

Use when:

- Authenticated data
- Real-time dashboards
- User-specific content

Strategy 5: Hybrid (Real-world default)

Mix strategies inside one page:

```
// Static product list
const products = await fetch('/api/products', {
  next: { revalidate: 60 }
})

// Dynamic user cart
const cart = await fetch('/api/cart', {
  cache: 'no-store'
})
```

This is where **App Router shines**.

---

3) Cache Invalidation Tools

These are the levers that matter in real apps:

Tag-based invalidation

```
revalidateTag('products')
```

Path-based invalidation

```
revalidatePath('/products')
```

---

4) Subtle but Important Rules

These trip people up:

1. Default behavior is NOT always obvious

- `fetch()` in RSC → cached by default
- But inside Server Actions → not cached

---

2. One `no-store` can make the whole route dynamic

If any fetch is:

```
cache: 'no-store'
```

→ Entire route becomes dynamic

---

3. Headers/Cookies break caching

Using:

```
cookies()
headers()
```

→ disables static optimization

---

4. Client Components don't cache data automatically

Caching is primarily a **server-side concern** in Next.js

---

5) Mental Model (What Actually Happens)

When a request hits:

1. Check **Full Route Cache**
2. If miss → execute server components
3. During execution:
    - `fetch()` hits **Data Cache**
4. Response cached again at route level
5. Client stores result in **Router Cache**

---

6) Real Example (Your Use Case Pattern)

You mentioned earlier:

> Product form (client) + product list (RSC)

Here's the correct strategy:

Product List (RSC)

```
await fetch('/api/products', {
  next: { tags: ['products'] }
})
```

After Adding Product

```
revalidateTag('products')
```

This gives:

- Fast cached reads
- Instant consistency after writes

Bottom Line

If you simplify everything:

- **Static** → fastest, least flexible
- **ISR** → good default
- **Tag revalidation** → best for real apps
- **Dynamic** → only when necessary